JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Process

java.lang.Object
    java.lang.Process

All Implemented Interfaces:

`Closeable`, `AutoCloseable`

```
public abstract class Process
extends Object
implements Closeable
```

`Process` provides control of native processes started by `ProcessBuilder.start` and `Runtime.exec`. The class provides methods for performing input from the process, performing output to the process, waiting for the process to complete, checking the exit status of the process, and destroying (killing) the process. The `ProcessBuilder.start()` and `Runtime.exec` methods create a native process and return an instance of a subclass of `Process` that can be used to control the process and obtain information about it.

The methods that create processes may not work well for special processes on certain native platforms, such as native windowing processes, daemon processes, Win16/DOS processes on Microsoft Windows, or shell scripts.

By default, the created process does not have its own terminal or console. All its standard I/O (i.e. stdin, stdout, stderr) operations will be redirected to the parent process, where they can be accessed via the streams obtained using the methods `getOutputStream()`, `getInputStream()`, and `getErrorStream()`. The I/O streams of characters and lines can be written and read using the methods `outputWriter()`, `outputWriter(Charset)`, `inputReader()`, `inputReader(Charset)`, `errorReader()`, and `errorReader(Charset)`. The parent process uses these streams to feed input to and get output from the process. Because some native platforms only provide limited buffer size for standard input and output streams, failure to promptly write the input stream or read the output stream of the process may cause the process to block, or even deadlock.

Where desired, process I/O can also be redirected using methods of the `ProcessBuilder` class.

There is no requirement that the process represented by a `Process` object execute asynchronously or concurrently with respect to the Java process that owns the `Process` object.

As of 1.5, `ProcessBuilder.start()` is the preferred way to create a `Process`.

Subclasses of Process should ensure that each overridden method invokes the superclass method. For example, if close is overridden, the subclass should ensure that `Process.close()` is called.

```
public class LoggingProcess extends java.lang.Process {
    ...
    @Override
    public void close() throws IOException  {
        try {
            super.close();
        } catch (IOException ex) {
            LOGGER.log(ex);
         } finally {
            LOGGER.log("process closed");
        }
    }
    ...
}
```

Subclasses of Process that wrap another Process instance should override and delegate the `onExit()` and `toHandle()` methods to provide a fully functional Process including the process id, information about the process, direct children, and direct children plus descendants of those children of the process. Delegating to the underlying Process or ProcessHandle is typically easiest and most efficient.

Resource Usage

Starting a process uses resources in both the invoking process and the invoked process and for the communication streams between them. The resources to control the process and for communication between the processes are retained until there are no longer any references to the Process or the input, error, and output streams or readers, or they have been closed. The Process close method closes all the streams and terminates the process to release the resources. Using try-with-resources will ensure the process is terminated when the try-with-resources block exits.

The process is not killed when there are no more references to the `Process` object, but rather the process continues executing asynchronously. The process implementation closes file descriptors and handles for streams that are no longer referenced to prevent leaking operating system resources. Processes that have terminated or been terminated are monitored and their resources released.

Streams should be closed when they are no longer needed, to avoid delaying releasing the operating system resources. `Try-with-resources` can be used to open and close the streams.

For example, to capture the output of a program known to produce some output and then exit:

```
List<String> capture(List<String> args) throws Exception {
    ProcessBuilder pb = new ProcessBuilder(args);
    try (Process process = pb.start();
         BufferedReader in = process.inputReader()) {
        List<String> captured = in.readAllLines();
        int status = process.waitFor();
        if (status != 0) {
            throw new RuntimeException("Process %d: %s failed with %d"
                        .formatted(process.pid(), args, status));
        }
        return captured;
    }
}
```

Stream resources (file descriptor or handle) are always paired; one in the invoking process and the other end of that connection in the invoked process. Closing a stream at either end terminates communication but does not have any direct effect on the other Process. The closing of the stream typically results in the other process exiting.

Destroying a process signals the operating system to terminate the process. It is up to the operating system to clean up and release the resources of that process. Typically, file descriptors and handles are closed. When they are closed, any connections to other processes are terminated and file descriptors and handles in the invoking process signal end-of-file or closed. Usually, that is seen as an end-of-file or an exception.

Since:

1.0

Constructor Summary

Constructors

Constructor	Description
`Process()`	Default constructor for Process.

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
Stream<ProcessHandle>	**children**()	Returns a snapshot of the direct children of the process.
void	**close**()	Closes all reader and writer streams and waits for the process to terminate.
Stream<ProcessHandle>	**descendants**()	Returns a snapshot of the descendants of the process.
abstract void	**destroy**()	Kills the process.
Process	**destroyForcibly**()	Kills the process forcibly.
final BufferedReader	**errorReader**()	Returns a BufferedReader connected to the standard error of the process.
final BufferedReader	**errorReader**(Charset charset)	Returns a BufferedReader connected to the standard error of this process using a Charset.
abstract int	**exitValue**()	Returns the exit value for the process.
abstract InputStream	**getErrorStream**()	Returns the input stream connected to the error output of the process.
abstract InputStream	**getInputStream**()	Returns the input stream connected to the normal output of the process.
abstract OutputStream	**getOutputStream**()	Returns the output stream connected to the

		normal input of the process.
ProcessHandle.Info	**info**()	Returns a snapshot of information about the process.
final BufferedReader	**inputReader**()	Returns a BufferedReader connected to the standard output of the process.
final BufferedReader	**inputReader** (Charset charset)	Returns a BufferedReader connected to the standard output of this process using a Charset.
boolean	**isAlive**()	Tests whether the process represented by this Process is alive.
CompletableFuture <Process>	**onExit**()	Returns a CompletableFuture<Process> for the termination of the Process.
final BufferedWriter	**outputWriter**()	Returns a BufferedWriter connected to the normal input of the process using the native encoding.
final BufferedWriter	**outputWriter** (Charset charset)	Returns a BufferedWriter connected to the normal input of the process using a Charset.
long	**pid**()	Returns the native process ID of the process.

boolean	**supportsNormalTerminatio**	Returns `true` if the implementation of `destroy()` is to normally terminate the process, Returns `false` if the implementation of `destroy` forcibly and immediately terminates the process.
ProcessHandle	**toHandle**()	Returns a ProcessHandle for the Process.
abstract int	**waitFor**()	Causes the current thread to wait, if necessary, until the process represented by this `Process` object has terminated.
boolean	**waitFor**(long timeout, TimeUnit unit)	Causes the current thread to wait, if necessary, until the process represented by this `Process` object has terminated, or the specified waiting time elapses.
boolean	**waitFor** (Duration duration)	Causes the current thread to wait, if necessary, until the process represented by this `Process` object has terminated, or the specified waiting duration elapses.

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Details

Process

```
public Process()
```

Default constructor for Process.

Method Details

close

```
public void close()
           throws IOException
```

Closes all reader and writer streams and waits for the process to terminate. This method is idempotent, if this `Process` has already been closed invoking this method has no effect.

If the data from the process input or error streams is needed, it must be read before calling this method. The contents of streams that have not been read to end of stream are lost, they are discarded or ignored.

If the process exit value is of interest, then the caller must wait for the process to terminate before calling this method.

Streams should be closed when no longer needed. Closing an already closed stream usually has no effect but is specific to the stream. If an `IOException` occurs when closing a stream it is thrown after the process has terminated. Exceptions thrown by closing the streams, if any, are added to the first `IOException` as suppressed exceptions.

After the streams are closed this method waits for the process to terminate. If interrupted while waiting the process is forcibly destroyed and this method continues to wait for the process to terminate. The interrupted status is re-asserted before this method returns or any `IOExceptions` are thrown.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

API Note:

Try-with-resources example to write text to a process, read back the response, and close the streams and process:

```
void main() {
    try (Process p = new ProcessBuilder("cat").start();
         var writer = p.outputWriter();
         var reader = p.inputReader()) {
        writer.write(haiku);
        writer.close();
        // Read all lines and print each
        reader.readAllLines()
                .forEach(IO::println);
        var status = p.waitFor();
        if (status != 0)
            throw new RuntimeException("unexpected process status: " + status)
    } catch (Exception e) {
        System.out.println("Process failed: " + e);
    }
}

static final String haiku = """
            Oh, the sunrise glow;
            Paddling with the river flow;
            Chilling still, go slow.
            """;
```

Implementation Requirements:

This method closes the process I/O streams and then waits for the process to terminate. If `waitFor()` is interrupted the process is forcibly destroyed and then `close()` waits for the process to terminate.

Implementation Note:

Concrete implementations that override this class are strongly encouraged to override this method and invoke the superclass `close` method.

Throws:

`IOException` - if closing any of the streams throws an exception

Since:

26

getOutputStream

```
public abstract OutputStream getOutputStream()
```

Returns the output stream connected to the normal input of the process. Output to the stream is piped into the standard input of the process represented by this `Process` object.

If the standard input of the process has been redirected using `ProcessBuilder.redirectInput` then this method will return a null output stream.

The output stream should be closed when it is no longer needed.

API Note:

When writing to both `getOutputStream()` and either `outputWriter()` or `outputWriter(Charset)`, `BufferedWriter.flush` should be called before writes to the `OutputStream`.

Implementation Note:

Implementation note: It is a good idea for the returned output stream to be buffered.

Returns:

the output stream connected to the normal input of the process

getInputStream

```
public abstract InputStream getInputStream()
```

Returns the input stream connected to the normal output of the process. The stream obtains data piped from the standard output of the process represented by this `Process` object.

If the standard output of the process has been redirected using `ProcessBuilder.redirectOutput` then this method will return a null input stream.

Otherwise, if the standard error of the process has been redirected using `ProcessBuilder.redirectErrorStream` then the input stream returned by this method will receive the merged standard output and the standard error of the process.

The input stream should be closed when it is no longer needed.

API Note:

Use either this method or an input reader but not both on the same `Process`. The input reader consumes and buffers bytes from the input stream. Bytes read from the input stream would not be seen by the reader and buffer contents are unpredictable.

Implementation Note:

Implementation note: It is a good idea for the returned input stream to be buffered.

Returns:

the input stream connected to the normal output of the process

getErrorStream

```
public abstract InputStream getErrorStream()
```

Returns the input stream connected to the error output of the process. The stream obtains data piped from the error output of the process represented by this `Process` object.

If the standard error of the process has been redirected using `ProcessBuilder.redirectError` or `ProcessBuilder.redirectErrorStream` then this method will return a null input stream.

The error stream should be closed when it is no longer needed.

API Note:

Use either this method or an error reader but not both on the same `Process`. The error reader consumes and buffers bytes from the error stream. Bytes read from the error stream would not be seen by the reader and the buffer contents are unpredictable.

Implementation Note:

Implementation note: It is a good idea for the returned input stream to be buffered.

Returns:

the input stream connected to the error output of the process

inputReader

```
public final BufferedReader inputReader()
```

Returns a `BufferedReader` connected to the standard output of the process. The `Charset` for the native encoding is used to read characters, lines, or stream lines from standard output.

This method delegates to `inputReader(Charset)` using the `Charset` named by the `native.encoding` system property. If the `native.encoding` is not a valid charset name or not supported the `Charset.defaultCharset()` is used.

The reader should be closed when it is no longer needed.

API Note:

Use either this method or the input stream but not both on the same `Process`. The input reader consumes and buffers bytes from the input stream. Bytes read from the input stream would not be seen by the reader and the buffer contents are unpredictable.

Returns:

a `BufferedReader` using the `native.encoding` if supported, otherwise, the `Charset.defaultCharset()`

Since:

17

inputReader

```
public final BufferedReader inputReader(Charset charset)
```

Returns a `BufferedReader` connected to the standard output of this process using a Charset. The `BufferedReader` can be used to read characters, lines, or stream lines of the standard output.

Characters are read by an InputStreamReader that reads and decodes bytes from this process `getInputStream()`. Bytes are decoded to characters using the `charset`; malformed-input and unmappable-character sequences are replaced with the charset's default replacement. The `BufferedReader` reads and buffers characters from the InputStreamReader.

The first call to this method creates the `BufferedReader`, if called again with the same `charset` the same `BufferedReader` is returned. It is an error to call this method again with a different `charset`.

If the standard output of the process has been redirected using `ProcessBuilder.redirectOutput` then the `InputStreamReader` will be reading from a null input stream.

The reader should be closed when it is no longer needed.

Otherwise, if the standard error of the process has been redirected using `ProcessBuilder.redirectErrorStream` then the input reader returned by this method will receive the merged standard output and the standard error of the process.

API Note:

Use either this method or the input stream but not both on the same `Process`. The input reader consumes and buffers bytes from the input stream. Bytes read from the input stream would not be seen by the reader and the buffer contents are unpredictable.

When the process has terminated, and the standard input has not been redirected, reading of the bytes available from the underlying stream is on a best effort basis and may be unpredictable.

Parameters:

`charset` - the `Charset` used to decode bytes to characters

Returns:

a `BufferedReader` for the standard output of the process using the `charset`

Throws:

`NullPointerException` - if the `charset` is `null`

`IllegalStateException` - if called more than once with different charset arguments

Since:

17

errorReader

```
public final BufferedReader errorReader()
```

Returns a `BufferedReader` connected to the standard error of the process. The `Charset` for the native encoding is used to read characters, lines, or stream lines from standard error.

This method delegates to `errorReader(Charset)` using the `Charset` named by the `native.encoding` system property. If the `native.encoding` is not a valid charset name or not supported the `Charset.defaultCharset()` is used.

The error reader should be closed when it is no longer needed.

API Note:

Use either this method or the error stream but not both on the same `Process`. The error reader consumes and buffers bytes from the error stream. Bytes read from the error stream would not be seen by the reader and the buffer contents are unpredictable.

Returns:

a `BufferedReader` using the `native.encoding` if supported, otherwise, the `Charset.defaultCharset()`

Since:

17

errorReader

```
public final BufferedReader errorReader(Charset charset)
```

Returns a `BufferedReader` connected to the standard error of this process using a Charset. The `BufferedReader` can be used to read characters, lines, or stream lines of the standard error.

Characters are read by an InputStreamReader that reads and decodes bytes from this process `getErrorStream()`. Bytes are decoded to characters using the `charset`; malformed-input and unmappable-character sequences are replaced with the charset's default replacement. The `BufferedReader` reads and buffers characters from the InputStreamReader.

The first call to this method creates the `BufferedReader`, if called again with the same `charset` the same `BufferedReader` is returned. It is an error to call this method again with a different `charset`.

If the standard error of the process has been redirected using `ProcessBuilder.redirectError` or `ProcessBuilder.redirectErrorStream` then the `InputStreamReader` will be reading from a null input stream.

The error reader should be closed when it is no longer needed.

API Note:

Use either this method or the error stream but not both on the same `Process`. The error reader consumes and buffers bytes from the error stream. Bytes read from the error stream would not be seen by the reader and the buffer contents are unpredictable.

When the process has terminated, and the standard error has not been redirected, reading of the bytes available from the underlying stream is on a best effort basis and may be unpredictable.

Parameters:

`charset` - the `Charset` used to decode bytes to characters

Returns:

a `BufferedReader` for the standard error of the process using the `charset`

Throws:

`NullPointerException` - if the `charset` is `null`

`IllegalStateException` - if called more than once with different charset arguments

Since:

17

outputWriter

```
public final BufferedWriter outputWriter()
```

Returns a `BufferedWriter` connected to the normal input of the process using the native encoding. Writes text to a character-output stream, buffering characters to provide for the efficient writing of single characters, arrays, and strings.

This method delegates to `outputWriter(Charset)` using the `Charset` named by the `native.encoding` system property. If the `native.encoding` is not a valid charset name or not supported the `Charset.defaultCharset()` is used.

The output writer should be closed when it is no longer needed.

Returns:

a `BufferedWriter` to the standard input of the process using the charset for the `native.encoding` system property

Since:

17

outputWriter

```
public final BufferedWriter outputWriter(Charset charset)
```

Returns a `BufferedWriter` connected to the normal input of the process using a Charset. Writes text to a character-output stream, buffering characters to provide for the efficient writing of single characters, arrays, and strings.

Characters written by the writer are encoded to bytes using `OutputStreamWriter` and the `Charset` are written to the standard input of the process represented by this `Process`. Malformed-input and unmappable-character sequences are replaced with the charset's default replacement.

The first call to this method creates the `BufferedWriter`, if called again with the same `charset` the same `BufferedWriter` is returned. It is an error to call this method again with a different `charset`.

If the standard input of the process has been redirected using `ProcessBuilder.redirectInput` then the `OutputStreamWriter` writes to a null output stream.

The output writer should be closed when it is no longer needed.

API Note:

A BufferedWriter writes characters, arrays of characters, and strings. Wrapping the `BufferedWriter` with a `PrintWriter` provides efficient buffering and formatting of primitives and objects as well as support for auto-flush on line endings. Call the `BufferedWriter.flush()` method to flush buffered output to the process.

When writing to both `getOutputStream()` and either `outputWriter()` or `outputWriter(Charset)`, BufferedWriter.flush should be called before writes to the `OutputStream`.

Parameters:

`charset` - the `Charset` to encode characters to bytes

Returns:

a `BufferedWriter` to the standard input of the process using the `charset`

Throws:

`NullPointerException` - if the `charset` is `null`

`IllegalStateException` - if called more than once with different charset arguments

Since:

17

waitFor

```
public abstract int waitFor()
                     throws InterruptedException
```

Causes the current thread to wait, if necessary, until the process represented by this `Process` object has terminated. This method returns immediately if the process has already terminated. If the process has not yet terminated, the calling thread will be blocked until the process exits.

Returns:

the exit value of the process represented by this `Process` object. By convention, the value `0` indicates normal termination.

Throws:

`InterruptedException` - if the current thread is interrupted by another thread while it is waiting, then the wait is ended and an `InterruptedException` is thrown.

waitFor

```
public boolean waitFor(long timeout,
                       TimeUnit unit)
                throws InterruptedException
```

Causes the current thread to wait, if necessary, until the process represented by this `Process` object has terminated, or the specified waiting time elapses.

If the process has already terminated then this method returns immediately with the value `true`. If the process has not terminated and the timeout value is

less than, or equal to, zero, then this method returns immediately with the value `false`.

Implementation Requirements:

The default implementation of this method polls the `exitValue` to check if the process has terminated.

Implementation Note:

Concrete implementations of this class are strongly encouraged to override this method with a more efficient implementation.

Parameters:

`timeout` - the maximum time to wait

`unit` - the time unit of the `timeout` argument

Returns:

`true` if the process has exited and `false` if the waiting time elapsed before the process has exited.

Throws:

`InterruptedException` - if the current thread is interrupted while waiting.

`NullPointerException` - if unit is null

Since:

1.8

waitFor

```
public boolean waitFor(Duration duration)
                throws InterruptedException
```

Causes the current thread to wait, if necessary, until the process represented by this `Process` object has terminated, or the specified waiting duration elapses.

If the process has already terminated then this method returns immediately with the value `true`. If the process has not terminated and the duration is not positive, then this method returns immediately with the value `false`.

Implementation Requirements:

The default implementation of this method polls the `exitValue` to check if the process has terminated.

Implementation Note:

Concrete implementations of this class are strongly encouraged to override this method with a more efficient implementation.

Parameters:

`duration` - the maximum duration to wait; if not positive, this method returns immediately.

Returns:

`true` if the process has exited and `false` if the waiting duration elapsed before the process has exited.

Throws:

`InterruptedException` - if the current thread is interrupted while waiting.

`NullPointerException` - if duration is null

Since:

24

exitValue

```
public abstract int exitValue()
```

Returns the exit value for the process.

Returns:

the exit value of the process represented by this `Process` object. By convention, the value `0` indicates normal termination.

Throws:

`IllegalThreadStateException` - if the process represented by this `Process` object has not yet terminated

destroy

```
public abstract void destroy()
```

Kills the process. Whether the process represented by this `Process` object is normally terminated or not is implementation dependent. Forcible process destruction is defined as the immediate termination of a process, whereas normal termination allows the process to shut down cleanly. If the process is not alive, no action is taken.

The `CompletableFuture` from `onExit()` is completed when the process has terminated.

destroyForcibly

```
public Process destroyForcibly()
```

Kills the process forcibly. The process represented by this `Process` object is forcibly terminated. Forcible process destruction is defined as the immediate termination of a process, whereas normal termination allows the process to shut down cleanly. If the process is not alive, no action is taken.

The `CompletableFuture` from `onExit()` is completed when the process has terminated.

Invoking this method on `Process` objects returned by `ProcessBuilder.start()` and `Runtime.exec(String)` forcibly terminate the process.

API Note:

The process may not terminate immediately. i.e. `isAlive()` may return true for a brief period after `destroyForcibly()` is called. This method may be chained to `waitFor()` if needed.

Implementation Requirements:

The default implementation of this method invokes `destroy()` and so may not forcibly terminate the process.

Implementation Note:

Concrete implementations of this class are strongly encouraged to override this method with a compliant implementation.

Returns:

the `Process` object representing the process forcibly destroyed

Since:

1.8

supportsNormalTermination

```
public boolean supportsNormalTermination()
```

Returns `true` if the implementation of `destroy()` is to normally terminate the process, Returns `false` if the implementation of `destroy` forcibly and immediately terminates the process.

Invoking this method on `Process` objects returned by `ProcessBuilder.start()` and `Runtime.exec(String)` return `true` or `false` depending on the platform implementation.

Implementation Requirements:

This implementation throws an instance of `UnsupportedOperationException` and performs no other action.

Returns:

`true` if the implementation of `destroy()` is to normally terminate the process; otherwise, `destroy()` forcibly terminates the process

Throws:

`UnsupportedOperationException` - if the Process implementation does not support this operation

Since:

9

isAlive

`public boolean isAlive()`

Tests whether the process represented by this `Process` is alive.

Returns:

`true` if the process represented by this `Process` object has not yet terminated.

Since:

1.8

pid

`public long pid()`

Returns the native process ID of the process. The native process ID is an identification number that the operating system assigns to the process.

Implementation Requirements:

The implementation of this method returns the process id as: `toHandle().pid()`.

Returns:

the native process id of the process

Throws:

`UnsupportedOperationException` - if the Process implementation does not support this operation

Since:

9

onExit

`public CompletableFuture<Process> onExit()`

Returns a `CompletableFuture<Process>` for the termination of the Process. The `CompletableFuture` provides the ability to trigger dependent functions or actions that may be run synchronously or asynchronously upon process termination. When the process has terminated the CompletableFuture is `completed` regardless of the exit status of the process.

Calling `onExit().get()` waits for the process to terminate and returns the Process. The future can be used to check if the process is done or to wait for it to terminate. Cancelling the CompletableFuture does not affect the Process.

Processes returned from `ProcessBuilder.start()` override the default implementation to provide an efficient mechanism to wait for process exit.

API Note:

Using `onExit` is an alternative to `waitFor` that enables both additional concurrency and convenient access to the result of the Process. Lambda expressions can be used to evaluate the result of the Process execution. If there is other processing to be done before the value is used then onExit is a convenient mechanism to free the current thread and block only if and when the value is needed.
For example, launching a process to compare two files and get a boolean if they are identical:

```
Process p = new ProcessBuilder("cmp", "f1", "f2").start();
Future<Boolean> identical = p.onExit().thenApply(p1 -> p1.exitValue() == 0
...
if (identical.get()) { ... }
```

, The process may be observed to have terminated with `isAlive()` before the ComputableFuture is completed and dependent actions are invoked.

Implementation Requirements:

This implementation executes `waitFor()` in a separate thread repeatedly until it returns successfully. If the execution of `waitFor` is interrupted, the thread's interrupted status is preserved.

When `waitFor()` returns successfully the CompletableFuture is completed regardless of the exit status of the process. This implementation may consume a lot of memory for thread stacks if a large number of processes are waited for concurrently.

External implementations should override this method and provide a more efficient implementation. For example, to delegate to the underlying process, it can do the following:

```
public CompletableFuture<Process> onExit() {
   return delegate.onExit().thenApply(p -> this);
}
```

Returns:

a new `CompletableFuture<Process>` for the Process

Since:

9

toHandle

`public ProcessHandle toHandle()`

Returns a ProcessHandle for the Process. `Process` objects returned by `ProcessBuilder.start()` and `Runtime.exec(String)` implement `toHandle` as the equivalent of `ProcessHandle.of(pid)`.

Implementation Requirements:

This implementation throws an instance of `UnsupportedOperationException` and performs no other action. Subclasses should override this method to provide a ProcessHandle for the process. The methods `pid()`, `info()`, `children()`, and `descendants()`, unless overridden, operate on the ProcessHandle.

Returns:

Returns a ProcessHandle for the Process

Throws:

`UnsupportedOperationException` - if the Process implementation does not support this operation

Since:

9

info

`public ProcessHandle.Info info()`

Returns a snapshot of information about the process.

A `ProcessHandle.Info` instance has accessor methods that return information about the process if it is available.

Implementation Requirements:

This implementation returns information about the process as: `toHandle().info()`.

Returns:

a snapshot of information about the process, always non-null

Throws:

`UnsupportedOperationException` - if the Process implementation does not support this operation

Since:

9

children

`public Stream<ProcessHandle> children()`

Returns a snapshot of the direct children of the process. The parent of a direct child process is the process. Typically, a process that is not alive has no children.

Note that processes are created and terminate asynchronously. There is no guarantee that a process is alive.

Implementation Requirements:

This implementation returns the direct children as: `toHandle().children()`.

Returns:

a sequential Stream of ProcessHandles for processes that are direct children of the process

Throws:

`UnsupportedOperationException` - if the Process implementation does not support this operation

Since:

9

descendants

`public Stream<ProcessHandle> descendants()`

Returns a snapshot of the descendants of the process. The descendants of a process are the children of the process plus the descendants of those children, recursively. Typically, a process that is not alive has no children.

Note that processes are created and terminate asynchronously. There is no guarantee that a process is alive.

Implementation Requirements:

This implementation returns all children as: `toHandle().descendants()`.

Returns:

a sequential Stream of ProcessHandles for processes that are descendants of the process

Throws:

`UnsupportedOperationException` - if the Process implementation does not support this operation

Since:

9

Report a bug or suggest an enhancement

For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.

Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.

All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.